Applications of Breadth-First Search

Lecture Notes with Algorithms, Worked Examples and Graph Diagrams

Data Structures and Algorithms
Undergraduate and GATE Preparation

Contents

1 Introduction to Breadth-First Search **3**
1.1 Basic BFS Algorithm . . . 3
1.2 Important Applications . . . 3

2 Connected Components Using BFS **3**
2.1 Definition . . . 4
2.2 Example Graph . . . 4
2.3 Step-by-Step Procedure . . . 4
2.4 Algorithm . . . 5

3 Shortest Path in an Unweighted Graph **5**
3.1 Definition . . . 5
3.2 Example Graph . . . 5
3.3 BFS Distance Calculation . . . 6
3.4 Shortest Path Algorithm . . . 7
3.5 Important Property . . . 7

4 Cycle Detection Using BFS **7**
4.1 Definition . . . 7
4.2 Example Graph . . . 8
4.3 Step-by-Step Explanation . . . 8
4.4 Cycle Detection Condition . . . 8
4.5 Algorithm . . . 9

5 Bipartite Graph Checking Using BFS **9**
5.1 Definition . . . 9
5.2 Example 1: Bipartite Graph . . . 9
5.3 BFS Two-Coloring . . . 10
5.4 Example 2: Non-Bipartite Graph . . . 10
5.5 Fundamental Theorem . . . 11
5.6 Algorithm . . . 11

6 Spanning Tree Using BFS **11**

6.1 Definition . . . 12
6.2 Example Graph . . . 12
6.3 BFS Traversal . . . 12
6.4 Number of Edges . . . 13
6.5 Algorithm . . . 13
6.6 Spanning Forest Formula . . . 13

7 Level and Distance Calculation **14**
7.1 Definition . . . 14
7.2 Example . . . 14
7.3 Level Table . . . 14
7.4 Algorithm . . . 14

8 Network Broadcasting Using BFS **15**
8.1 Concept . . . 15
8.2 Example . . . 15

9 Comparison of BFS Applications **16**

10 Important Formulas for Examinations **16**

11 Solved Numerical Problems **16**

12 Practice Questions **18**

13 Answers to Practice Questions **19**

1. Introduction to Breadth-First Search

Breadth-First Search (BFS) is a graph traversal algorithm that explores vertices level by level.

It first visits all vertices directly connected to the source vertex, then vertices at distance two, followed by distance three, and so on.

BFS uses a **queue**, which follows the First-In-First-Out (FIFO) principle.

1.1 Basic BFS Algorithm

```python
from collections import deque

def bfs(adj, source):
    visited = {source}
    queue = deque([source])
    order = []

    while queue:
        u = queue.popleft()
        order.append(u)

        for v in adj[u]:
            if v not in visited:
                visited.add(v)
                queue.append(v)

    return order
```

For adjacency-list representation,

$$\boxed{\text{Time Complexity} = O(V + E)}$$

The auxiliary space complexity is

$$\boxed{O(V)}.$$

1.2 Important Applications

1. Connected components
2. Shortest paths in unweighted graphs
3. Cycle detection
4. Bipartite graph checking
5. BFS spanning trees
6. Level and distance calculation
7. Network broadcasting

2. Connected Components Using BFS

2.1 Definition

A **connected component** is a maximal connected subgraph of an undirected graph in which every pair of vertices is connected by a path.

BFS identifies connected components by starting a new traversal whenever an unvisited vertex is found.

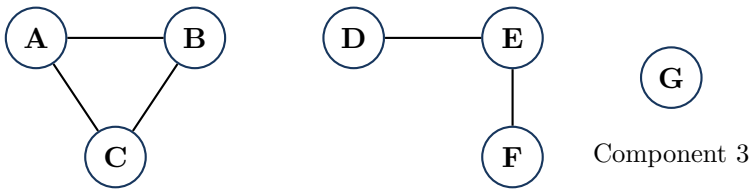
2.2 Example Graph

Consider an undirected graph with

$$V = \{A, B, C, D, E, F, G\}$$

and

$$E = \{AB, AC, BC, DE, EF\}.$$



2.3 Step-by-Step Procedure

Step 1: Start BFS from vertex A.

$$Q = [A]$$

Remove A and enqueue its unvisited neighbors:

$$Q = [B, C].$$

Process B and C.

The first component is

$$C_1 = \{A, B, C\}.$$

Step 2: Find another unvisited vertex.

Vertex D is unvisited.

Start BFS:

$$D \to E \to F.$$

Thus,

$$C_2 = \{D, E, F\}.$$

Step 3: Vertex G remains unvisited.

Since G is isolated,

$$C_3 = \{G\}.$$

$$\boxed{\text{Number of Connected Components} = 3}$$

2.4 Algorithm

```python
from collections import deque

def connected_components(adj):
    visited = set()
    components = []

    for start in adj:
        if start in visited:
            continue

        queue = deque([start])
        visited.add(start)
        component = []

        while queue:
            u = queue.popleft()
            component.append(u)

            for v in adj[u]:
                if v not in visited:
                    visited.add(v)
                    queue.append(v)

        components.append(component)

    return components
```

Observation: Each BFS started from an unvisited vertex discovers exactly one connected component.

3. Shortest Path in an Unweighted Graph

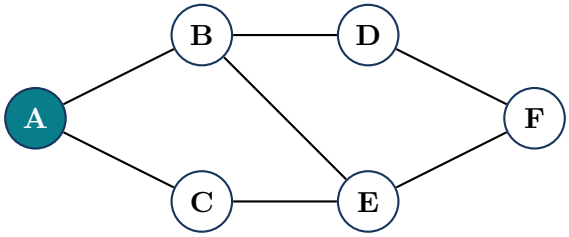
3.1 Definition

A shortest path between two vertices is a path containing the minimum number of edges connecting them.
BFS finds shortest paths in an unweighted graph, where every edge has equal cost.

3.2 Example Graph

Consider the following graph.

$$E = \{AB, AC, BD, BE, CE, DF, EF\}.$$



Find the shortest path from A to F.

3.3 BFS Distance Calculation

Starting from A:

$$d[A] = 0.$$

The immediate neighbors are B and C:

$$d[B] = d[C] = 1.$$

Next, vertices D and E are reached:

$$d[D] = d[E] = 2.$$

Finally, vertex F is discovered:

$$d[F] = 3.$$

Vertex	Distance from A	Parent
A	0	–
B	1	A
C	1	A
D	2	B
E	2	B
F	3	D

The table assumes neighbors are visited alphabetically.

Following the parent pointers:

$$F \leftarrow D \leftarrow B \leftarrow A.$$

Therefore,

$$\boxed{A \rightarrow B \rightarrow D \rightarrow F}$$

is one shortest path.

$$\boxed{\text{Shortest Path Length} = 3}$$

The graph also contains another shortest path $A \to C \to E \to F$.

3.4 Shortest Path Algorithm

```python
from collections import deque

def bfs_shortest_path(adj, source, target):
    queue = deque([source])
    distance = {source: 0}
    parent = {source: None}

    while queue:
        u = queue.popleft()

        for v in adj[u]:
            if v not in distance:
                distance[v] = distance[u] + 1
                parent[v] = u
                queue.append(v)

    if target not in distance:
        return None, None

    path = []
    current = target

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()
    return distance[target], path
```

3.5 Important Property

For any edge (u, v) in an undirected graph,

$$\boxed{|d[u] - d[v]| \leq 1}$$

where $d[u]$ and $d[v]$ are finite BFS distances from the same source.

BFS does not generally solve shortest paths in weighted graphs.

4. Cycle Detection Using BFS

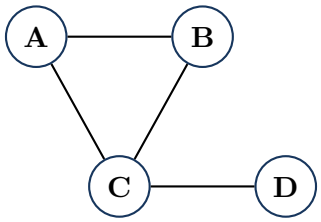
4.1 Definition

A cycle is a path that starts and ends at the same vertex without repeating other vertices or edges.
In an undirected graph, BFS can detect cycles using a parent array.

4.2 Example Graph

Consider

$$E = \{AB, BC, CA, CD\}.$$



4.3 Step-by-Step Explanation

Start BFS at A.

Step 1:

Visit A and discover B and C.

Their parent is A.

$$\text{parent}[B] = A$$

$$\text{parent}[C] = A.$$

Step 2:

Process B.

Its neighbors are A and C.

A is its parent, so edge BA is ignored for cycle detection.

C is already visited, and

$$C \neq \text{parent}[B].$$

Therefore, a cycle exists.

The cycle is

$$\boxed{A \to B \to C \to A}.$$

4.4 Cycle Detection Condition

For an undirected graph, if BFS processes an edge (u, v) such that

$$v \text{ is already visited}$$

and

$$\boxed{v \neq \text{parent}[u]},$$

then the graph contains a cycle.

4.5 Algorithm

```python
from collections import deque

def has_cycle_undirected(adj):
    visited = set()

    for start in adj:
        if start in visited:
            continue

        queue = deque([start])
        visited.add(start)
        parent = {start: None}

        while queue:
            u = queue.popleft()

            for v in adj[u]:
                if v not in visited:
                    visited.add(v)
                    parent[v] = u
                    queue.append(v)

                elif parent[u] != v:
                    return True

    return False
```

Important:
This parent-based cycle detection algorithm is for simple undirected graphs.
Directed graph cycle detection requires different methods, such as DFS recursion states or Kahn's topological sorting algorithm.

5. Bipartite Graph Checking Using BFS

5.1 Definition

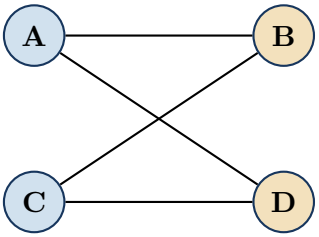
A graph is called **bipartite** if its vertices can be divided into two disjoint sets U and W such that every edge connects a vertex in U to a vertex in W.
No edge connects two vertices within the same set.

BFS checks bipartiteness through two-coloring.

5.2 Example 1: Bipartite Graph

Consider

$$E = \{AB, AD, CB, CD\}.$$



$U = \{A, C\}$ $W = \{B, D\}$

5.3 BFS Two-Coloring

Assign color 0 to A:

$$\text{color}[A] = 0.$$

Assign color 1 to its neighbors B and D:

$$\text{color}[B] = 1, \qquad \text{color}[D] = 1.$$

Next, both B and D require C to have color 0:

$$\text{color}[C] = 0.$$

Thus,

$$U = \{A, C\}$$

and

$$W = \{B, D\}.$$

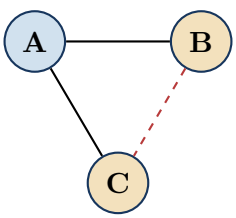
No edge joins two vertices of the same color.

The graph is bipartite

5.4 Example 2: Non-Bipartite Graph

Consider a triangle:

$$E = \{AB, BC, CA\}.$$



Start BFS from A.

Assign

$$\text{color}[A] = 0.$$

Both neighbors receive color 1:

$$\text{color}[B] = \text{color}[C] = 1.$$

But an edge exists between B and C.

Two adjacent vertices have the same color.

Therefore,

$$\boxed{\text{The graph is not bipartite}}.$$

5.5 Fundamental Theorem

> An undirected graph is bipartite if and only if it contains no cycle of odd length.

A triangle contains a cycle of length 3 and is therefore not bipartite.

5.6 Algorithm

```python
from collections import deque

def is_bipartite(adj):
    color = {}

    for start in adj:
        if start in color:
            continue

        color[start] = 0
        queue = deque([start])

        while queue:
            u = queue.popleft()

            for v in adj[u]:
                if v not in color:
                    color[v] = 1 - color[u]
                    queue.append(v)

                elif color[v] == color[u]:
                    return False

    return True
```

$$\boxed{T(V, E) = O(V + E)}$$

6. Spanning Tree Using BFS

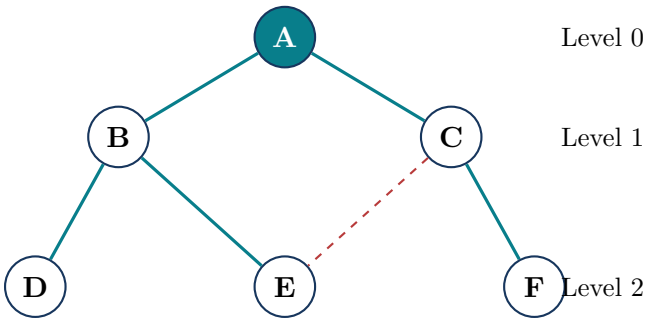
6.1 Definition

A BFS spanning tree contains every vertex of a connected undirected graph and includes the edges through which BFS first discovers each vertex.
Vertices in a BFS spanning tree are organized by their distance from the source.

6.2 Example Graph

Consider

$$E = \{AB, AC, BD, BE, CE, CF\}.$$



The tree edges are

$$\{AB, AC, BD, BE, CF\}.$$

The non-tree edge is

$$\{CE\}.$$

6.3 BFS Traversal

Starting from A:

$$\text{Level } 0 = \{A\}$$

$$\text{Level } 1 = \{B, C\}$$

$$\text{Level } 2 = \{D, E, F\}.$$

Thus, the BFS traversal is

$$\boxed{A, B, C, D, E, F}$$

when neighbors are examined alphabetically.

6.4 Number of Edges

A spanning tree has exactly $V - 1$ edges.

Here,

$$V = 6.$$

Therefore,

$$\boxed{V - 1 = 5}.$$

6.5 Algorithm

```python
from collections import deque

def bfs_spanning_forest(adj):
    visited = set()
    tree_edges = []

    for start in adj:
        if start in visited:
            continue

        queue = deque([start])
        visited.add(start)

        while queue:
            u = queue.popleft()

            for v in adj[u]:
                if v not in visited:
                    visited.add(v)
                    tree_edges.append((u, v))
                    queue.append(v)

    return tree_edges
```

6.6 Spanning Forest Formula

For an undirected graph with V vertices and C connected components,

$$\boxed{\text{BFS Forest Edges} = V - C}$$

For a connected graph,

$$C = 1,$$

so

$$\boxed{\text{Tree Edges} = V - 1}.$$

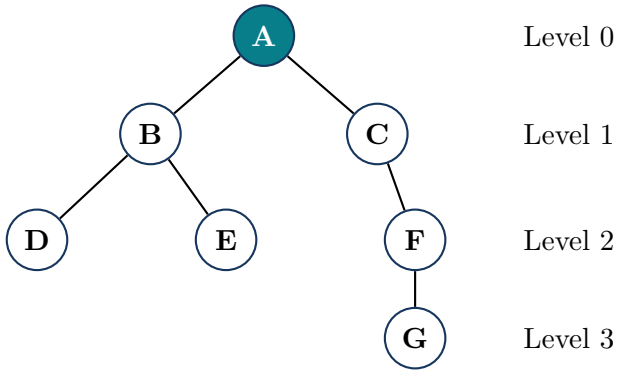
7. Level and Distance Calculation

7.1 Definition

The level of a vertex in BFS is the minimum number of edges required to reach that vertex from the source vertex.
The source has level 0.

7.2 Example

Consider the following graph.



7.3 Level Table

Vertex	BFS Level
A	0
B	1
C	1
D	2
E	2
F	2
G	3

The recurrence for BFS discovery is

$$\boxed{d[v] = d[u] + 1}$$

when v is first discovered from u.

7.4 Algorithm

```
from collections import deque

def bfs_levels(adj, source):
    level = {source: 0}
    queue = deque([source])

    while queue:
        u = queue.popleft()
```

```
9
10          for v in adj[u]:
11              if v not in level:
12                  level[v] = level[u] + 1
13                  queue.append(v)
14
15      return level
```

8. Network Broadcasting Using BFS

8.1 Concept

BFS models the spread of information through an unweighted network when each transmission takes one time unit and all informed vertices can transmit in parallel during the next time step.

Examples include:

- Information dissemination in networks
- Message propagation in social networks
- Flooding-based communication
- Rumor or infection-spread models

8.2 Example

Consider the network in Section 7, with vertex A as the information source.

At time $t = 0$, only A knows the information.

At time $t = 1$, vertices B and C receive it.

At time $t = 2$, vertices D, E and F receive it.

At time $t = 3$, vertex G receives it.

Time	Newly Informed Vertices
0	A
1	B, C
2	D, E, F
3	G

Therefore, all vertices are informed after 3 transmission rounds.

$$\boxed{\text{Broadcast Completion Time} = 3}$$

In this idealized model, the broadcast completion time equals the maximum finite BFS distance from the source, provided all network vertices are reachable.

$$\boxed{T_{\text{broadcast}} = \max_{v \in V} d[v]}$$

This quantity is the eccentricity of the source vertex in a connected, unweighted, undirected graph.

9. Comparison of BFS Applications

Application	Graph Type	Main BFS Technique	Complexity
Connected components	Undirected	Repeated BFS from unvisited vertices	$O(V+E)$
Shortest paths	Unweighted	Distance and parent arrays	$O(V+E)$
Cycle detection	Undirected	Visited vertices and parent checking	$O(V+E)$
Bipartite checking	Undirected	Two-coloring by BFS	$O(V+E)$
Spanning tree	Undirected	Record discovery edges	$O(V+E)$
Level calculation	Both	Distance increment	$O(V+E)$
Network broadcasting	Unweighted	Level-wise propagation	$O(V+E)$

10. Important Formulas for Examinations

1. BFS Forest Edges

$$\boxed{V - C}$$

where C is the number of connected components of an undirected graph.

2. BFS Spanning Tree Edges

$$\boxed{V - 1}$$

for a connected graph.

3. Non-Tree Edges

$$\boxed{E - V + C}$$

for an undirected graph with E edges.

4. BFS Distance Update

$$\boxed{d[v] = d[u] + 1}$$

when v is first discovered from u.

5. Undirected Cycle Detection

A visited neighbor other than the BFS parent indicates a cycle.

6. Bipartite Graph Condition

$$\boxed{\text{No Odd-Length Cycle}}$$

7. BFS Level Property

For every undirected edge (u, v),

$$\boxed{|d[u] - d[v]| \leq 1}$$

when the distances are finite.

11. Solved Numerical Problems

Problem 1: Connected Components

An undirected graph has 20 vertices. A BFS forest contains 16 tree edges.

Determine the number of connected components.

Solution:

$$V - C = 16.$$

$$20 - C = 16.$$

Therefore,

$$\boxed{C = 4}.$$

Problem 2: Non-Tree Edges

An undirected graph contains 15 vertices, 22 edges and 3 connected components.

Determine the number of non-tree edges in a BFS forest.

Solution:

Tree edges:

$$V - C = 15 - 3 = 12.$$

Non-tree edges:

$$E - (V - C) = 22 - 12.$$

$$\boxed{10}.$$

Problem 3: Shortest Path

A BFS traversal discovers vertex X at level 4.

Find the minimum number of edges from the source to X.

Solution:

BFS level equals shortest-path distance.

$$\boxed{d[X] = 4}.$$

Problem 4: Bipartite Graph

An undirected graph contains a cycle of length 7.

Can the graph be bipartite?

Solution:

A bipartite graph cannot contain an odd-length cycle.

Since 7 is odd,

$$\boxed{\text{The graph is not bipartite}}.$$

Problem 5: BFS Forest

An undirected graph has 12 vertices and 17 edges.

A complete BFS traversal produces a forest with 10 tree edges.

Calculate:

(a) Number of connected components

(b) Number of non-tree edges

Solution:

$$V - C = 10.$$

$$12 - C = 10.$$

Therefore,

$$\boxed{C = 2}.$$

Non-tree edges:

$$17 - 10 = \boxed{7}.$$

12. Practice Questions

Q1. An undirected graph has 25 vertices and 4 connected components. How many edges are in its BFS forest?

Q2. A connected undirected graph has 18 vertices and 28 edges. Find the number of non-tree edges in a BFS spanning tree.

Q3. Explain how BFS identifies a cycle in an undirected graph.

Q4. Why is BFS more suitable than ordinary DFS for finding shortest paths in an unweighted graph?

Q5. Explain how BFS uses two-coloring to check whether a graph is bipartite.

Q6. A graph contains a cycle of length 9. Is it bipartite?

Q7. A source vertex has eccentricity 5 in a connected, unweighted network. Under the parallel unit-time broadcast model, how many rounds are required to inform all vertices?

13. Answers to Practice Questions

Answer 1

$$V - C = 25 - 4 = \boxed{21}.$$

Answer 2

Spanning tree edges:

$$18 - 1 = 17.$$

Non-tree edges:

$$28 - 17 = \boxed{11}.$$

Answer 3

BFS maintains a parent for each vertex. If it encounters an already visited neighbor different from the current vertex's parent, an undirected cycle has been detected.

Answer 4

BFS explores vertices in nondecreasing order of their distance from the source. Therefore, its first discovery of a vertex gives a shortest path in an unweighted graph.

Ordinary DFS does not provide this guarantee.

Answer 5

Assign color 0 to an unvisited source. Assign each undiscovered neighbor the opposite color.

If an edge connects two vertices with the same color, the graph is not bipartite.

Answer 6

Since the graph contains an odd-length cycle,

$$\boxed{\text{Not bipartite}}.$$

Answer 7

Broadcast time equals source eccentricity under the specified model.

$$\boxed{5 \text{ rounds}}.$$